

NCAA March Madness Prediction System

Mixture-of-Experts, Temporal Modeling, and Prediction Market Trading

Roddy Huang, Yumeng Jin
Cornell University

April 2026

Abstract

We built a probabilistic prediction system for the NCAA Men’s Basketball Tournament targeting both the Kaggle March Machine Learning Mania competition and simulated trading on the Kalshi prediction market. We implemented ten models across three tiers of complexity and evaluated them via leave-one-tournament-out cross-validation on 11 seasons (2014–2025). The best single model, a multi-feature logistic regression using Barttorvik continuous efficiency ratings, achieved a Brier score of 0.159, beating the Vegas benchmark (~ 0.175) and our original target of < 0.170 . A simulated Kalshi trading strategy using fractional Kelly sizing returned 20.6% ROI with a 0.88 Sharpe ratio over the same 11 seasons. The central finding is that feature quality dominates model complexity: switching from ordinal rankings to continuous efficiency metrics produced a larger improvement than any architectural change we tried, including gradient boosting, mixture-of-experts, and Transformer sequence models.

1 Introduction

The NCAA Division I Men’s Basketball Tournament is a 68-team single-elimination event that produces 67 games over three weeks each March. The format makes it one of the most unpredictable sporting events in the world. One bad shooting night ends your season. Gonzaga can go 32–1 in the regular season and lose in the round of 32 to a 12-seed.

The Kaggle March Machine Learning Mania competition asks participants to predict $P(\text{Team}_i \text{ beats Team}_j)$ for every possible matchup among tournament teams. Submissions are scored by Brier score:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (p_i - o_i)^2 \quad (1)$$

where p_i is the predicted probability and $o_i \in \{0, 1\}$ is the outcome. Lower is better. A coin-flip prediction (0.5 for everything) gives 0.250. Predicting from seed alone gives about 0.200. The Vegas line sits around 0.175. Top Kaggle entries get below 0.170.

A consistent finding from 12 years of competition history is that simple models with good features beat complex models with mediocre features [Pomeroy, 2004]. Most winners use logistic regression or Elo with KenPom efficiency ratings. We came into this project thinking we could improve on that with mixture-of-experts and Transformer architectures. We were partly right and mostly wrong, and the story of why is the main content of this paper.

Our project had two goals. The first was prediction: build the best Brier score we could and generate a Kaggle submission. The second was trading: design and backtest a strategy for the Kalshi prediction market, where NCAA games trade as binary contracts with 0% maker fee. Both goals forced us to think carefully about calibration, because a model that’s good at ranking teams by strength is not the same as one that outputs well-calibrated probabilities.

2 Data

2.1 Kaggle dataset

The Kaggle competition provides about 35 CSV files per edition. We used the Men’s division data spanning 2003–2026. The core tables are regular-season box scores (124,530 games with full shooting, rebounding, and turnover statistics), tournament results (2,586 games), seeds (68 teams per year), and the Massey ordinals file, which logs daily rankings from over 100 independent rating systems for every D1 team. The Massey file alone is 5.9 million rows.

From the Massey ordinals we extracted 9 systems with broad historical coverage: POM (KenPom), SAG (Sagarin), MOR (Massey composite), DOL (Dolphin), COL (Colley), AP, USA Today, WOL (Wolfe), and RPI. All of these are integer ordinal ranks from 1 to 363. This turns out to matter a lot, and we’ll come back to it.

2.2 Barttorvik

This was the data source that changed everything for us. Barttorvik [Barttorvik, 2017] publishes free tempo-adjusted efficiency ratings for every D1 team: adjusted offensive efficiency (AdjOE, points per 100 possessions), adjusted defensive efficiency (AdjDE), net rating, Barthag (a Pythagorean win probability estimate), and tempo. These are all continuous values.

We built a scraping pipeline that fetches T-Rank data for 2014–2025, handles the site’s JavaScript browser verification, and fuzzy-matches team names to Kaggle’s TeamID system using the `MTeamSpellings.csv` mapping. The scraper caches results to avoid repeated requests. The team-name matching was more annoying than it should have been (“Miami FL” vs “Miami (FL)” vs “Miami”, for instance) but `rapidfuzz` handled most of it after we wrote a manual override table for the worst offenders.

2.3 Box-score-derived features

From the regular-season detailed results we computed two sets of features. First, Dean Oliver’s Four Factors [Oliver, 2004]:

$$\text{eFG\%} = (\text{FGM} + 0.5 \times \text{FGM3}) / \text{FGA} \quad (2)$$

$$\text{TO\%} = \text{TO} / (\text{FGA} + 0.44 \times \text{FTA} + \text{TO}) \quad (3)$$

$$\text{OR\%} = \text{OR} / (\text{OR} + \text{Opp DR}) \quad (4)$$

$$\text{FT Rate} = \text{FTM} / \text{FGA} \quad (5)$$

We compute both offensive and defensive versions for each team, giving 8 per-team averages.

Second, we built our own tempo-adjusted efficiency metric from box scores, using 50 iterations of a strength-of-schedule fixed-point adjustment. The idea was to replicate what KenPom does without paying \$25 for a subscription. It didn’t work well (Brier 0.225 vs 0.196 for POM ordinal rank), because our SOS model is too crude compared to the regression frameworks that KenPom and Barttorvik use. We kept the features in the pipeline anyway since they’re free and XGBoost can decide whether to use them.

3 Feature Engineering

Each tournament matchup is represented as a feature vector encoding the difference between Team A and Team B across multiple axes. We order matchups canonically (lower TeamID = Team A), which gives a nearly balanced label distribution ($P(\text{Team A wins}) = 0.512$).

The full feature set has 53 columns organized into four layers:

Layer 1: Team ratings. Seed numbers, ordinal rank differences from the 9 Massey systems, and the 5 Barttorvik continuous features (AdjOE/AdjDE/NetRtg/Barthag/tempo differences). These are pre-tournament snapshots taken at day 133, just before Selection Sunday.

Layer 2: Efficiency. Our self-computed tempo-adjusted offensive and defensive efficiency, net efficiency, and tempo differences.

Layer 3: Box score. All 8 Four Factors differences (4 offensive + 4 defensive) computed as season averages.

Layer 4: Context. Rolling 10-game momentum (win percentage and margin), coaching tournament experience (number of prior NCAA appearances), and for the trading module, market-implied probability.

Not all features are available for all teams. The AP and USA Today polls only rank 25 teams, leaving about half the tournament field with missing values in those columns. The algorithmic rating systems (POM, SAG, MOR, Barttorvik) rank every D1 team and don't have this problem. We handle missingness by dropping poll-based columns from linear models and relying on native NaN handling in tree-based methods.

4 Models

We implemented ten models across three tiers. Table 1 summarizes all of them; the rest of this section focuses on the interesting ones.

Table 1: All models with LOTO Brier scores (2014–2025, 11 folds).

Tier	Model	Brier
5*Baseline	Seed Logistic	0.199
	KenPom (POM rank) Logistic	0.196
	Barttorvik (net rating) Logistic	0.169
	Efficiency Logistic	0.225
	Elo (MOV-adjusted)	0.221
4*Advanced	Multi-Feature Logistic (21 features)	0.159
	XGBoost (24 curated features)	0.172
	Mixture of Experts (K=4)	0.185
	Transformer (2-layer, pretrained)	0.196
Meta	Weighted Ensemble	0.161
	Market-Calibrated Ensemble ($\alpha=0.7$)	0.159

4.1 The ordinal rank ceiling

Our first two weeks were spent trying to beat 0.196. That's the score you get from a logistic regression on KenPom's ordinal rank difference. We tried XGBoost on 53 features (0.214, worse), Mixture-of-Experts (0.257, much worse before scaling fix), and a Transformer (0.245, embarrassingly worse). All of them lost to a one-variable logistic regression.

The problem was not the models. The problem was the features. Integer ranks from 1 to 363 throw away the continuous efficiency information that actually distinguishes teams. The gap between #1 and #5 might be 15 points of net efficiency, or it might be 2. Rank 5 doesn't tell you which. Once we scraped Barttorvik's continuous ratings and plugged them into the same logistic model, the score dropped from 0.196 to 0.169 overnight. That single-feature improvement was larger than anything we achieved through model architecture.

4.2 Multi-Feature Logistic

Our best single model is a logistic regression with StandardScaler on 21 selected features: seed difference, POM rank difference, all 5 Barttorvik differences, 4 self-computed efficiency differences, 8 Four Factors differences, 2 momentum features, and coaching experience. Regularized with $C = 0.5$. Brier 0.159.

We believe the logistic form works here because the relationship between team quality difference and win probability really is approximately sigmoidal. The log-odds of winning scale roughly linearly with the efficiency gap, which is exactly what logistic regression assumes. Once you feed it good continuous features, there isn't much room for nonlinear models to improve.

4.3 XGBoost

Our initial XGBoost scored 0.214 on 53 features. Worse than one-variable logistic regression. The issue was a combination of feature redundancy (27 of the 53 columns were highly correlated rank values from 9 different systems) and insufficient regularization for a training set of only ~ 670 games per LOTO fold.

After adding the 5 Barttorvik features, dropping 29 redundant rank columns, and tightening regularization (depth $4 \rightarrow 3$, estimators $300 \rightarrow 150$, min_child_weight 5, L1/L2 penalties), XGBoost came down to 0.172. Still not as good as the logistic model on the same features (0.159), which says something about how forgiving logistic regression is on small datasets versus how hungry gradient boosting is for data.

4.4 Mixture of Experts

The MoE uses a gating network to route each matchup to one of four specialized logistic regression experts, initialized by seed gap: Chalk ($|\Delta s| \geq 8$), Competitive ($4 \leq |\Delta s| < 8$), Toss-up ($|\Delta s| < 4$), and a catch-all. Training uses 10 iterations of EM.

The first version scored 0.257 because we forgot to scale the features. POM rank differences range from -250 to $+250$ while momentum lives on $[0, 1]$, and LBFGS can't converge when the gradient is dominated by a single feature. Adding StandardScaler brought it to 0.185 after Barttorvik integration. The experts do learn different behavior: the Chalk expert is very confident on 1-vs-16 games while the Toss-up expert hedges on 8-vs-9 matchups. But the gating doesn't add enough over a single logistic model to justify the complexity.

4.5 Transformer

We encode each team's regular season as a sequence of 13-dimensional game embeddings (opponent rank, margin, location, FG%, 3P%, FT%, rebound rate, assists, turnovers, steals, blocks, win indicator, normalized day). A 2-layer Transformer encoder with 32-dim embeddings produces a team embedding from the CLS token, and a matchup MLP predicts $P(A \text{ wins})$ from the concatenated embeddings.

The first version (4 layers, 64-dim, trained only on 736 tournament games) scored 0.245. The redesigned version (2 layers, 32-dim, $\sim 8K$ parameters) with a three-phase training pipeline scored 0.196:

1. Pretrain the encoder on $\sim 5K$ regular-season game pairs per season.
2. Freeze the encoder, train the matchup MLP on tournament data.
3. Unfreeze with $0.1 \times$ learning rate for joint fine-tuning.

Pretraining was the difference between a model that learned nothing and one that matches KenPom Logistic. But the Transformer still can't compete with the Barttorvik-based models because its game embeddings don't include cross-team adjusted efficiency. It knows a team's FG% and turnover rate against each opponent, but it doesn't know what those numbers mean

relative to how good the opponent’s defense is. Injecting Barttorvik features into the game embeddings is a natural next step that we didn’t have time to pursue.

4.6 Ensemble and market calibration

The weighted ensemble optimizes blend weights per LOTO fold on inner leave-one-season-out predictions. It puts 80–95% of the weight on Multi-Feature Logistic in every fold, yielding 0.161 Brier. The other models contribute almost no unique signal once Barttorvik features are in the mix.

We also tested Bayesian fusion with synthetic market prices via the formula from our proposal:

$$p_{\text{final}} = \frac{\hat{p}^\alpha \cdot p_{\text{mkt}}^{1-\alpha}}{\hat{p}^\alpha \cdot p_{\text{mkt}}^{1-\alpha} + (1 - \hat{p})^\alpha \cdot (1 - p_{\text{mkt}})^{1-\alpha}}$$

Grid-searching $\alpha \in [0.3, 1.0]$ gives an optimum at $\alpha = 0.7$ with Brier 0.1587, essentially unchanged from 0.1593 without calibration. The market signal (synthetic, 70% correlated with our model) adds almost nothing. This is consistent with the ensemble result: once the logistic model has Barttorvik data, there’s little room for improvement from additional signal sources.

5 Prediction Results

5.1 LOTO backtest

Table 2 breaks down the top 4 models by season.

Table 2: Per-season Brier scores for selected models (2014–2025). Bold = best model that season.

Season	MultiFeature	Barttorvik	XGBoost	Seed
2014	0.175	0.172	0.181	0.223
2015	0.161	0.157	0.165	0.178
2016	0.165	0.173	0.169	0.204
2017	0.145	0.167	0.152	0.184
2018	0.175	0.193	0.175	0.208
2019	0.130	0.134	0.147	0.179
2021	0.175	0.189	0.195	0.230
2022	0.171	0.175	0.183	0.220
2023	0.183	0.179	0.198	0.209
2024	0.133	0.157	0.152	0.196
2025	0.138	0.163	0.142	0.164
Mean	0.159	0.169	0.172	0.199

Year-to-year variance is large. The standard deviation across seasons is about 0.02 for every model, which is comparable to the gap between the best and worst models. 2019 was unusually easy (all models scored well; the bracket played out close to seed expectations) while 2021 was unusually hard (first post-COVID tournament, unusual rosters and scheduling). A single 16-over-1 upset costs about 0.01 Brier for the entire season.

5.2 2026 predictions

We trained the Multi-Feature Logistic on all 2014–2025 data and generated predictions for the 2026 tournament field. Some notable model opinions: Arizona over LIU Brooklyn at 99.8% (straightforward chalk), but Kentucky at only 28.3% against Santa Clara (the model sees a

7-seed vs 10-seed where the 10-seed has better AdjEM), and BYU at 34.7% against NC State (an 11-seed with stronger Barttorvik metrics than the 6-seed). These are the kinds of picks that make bracket pools interesting.

We generated the full Kaggle submission file (132,133 matchup probabilities) and a set of 36 trading signals for the Kalshi dry-run.

6 Trading Strategy

6.1 Design

We built a complete Kalshi trading module with signal generation, position sizing, risk management, and historical backtesting. The strategy trades when the absolute model-market disagreement exceeds a threshold τ : buy Yes if $p_{\text{model}} > p_{\text{market}} + \tau$, buy No otherwise.

Positions are sized by quarter-Kelly. For a binary contract priced at c :

$$f = 0.25 \cdot \frac{p_{\text{model}} - c}{1 - c}$$

We cap each position at 5% of the bankroll and total exposure at 20%. Each position is also capped at 100 contracts to reflect realistic Kalshi liquidity for NCAA markets.

6.2 Synthetic market

We couldn't get historical Kalshi odds (they don't publish them), so we built a synthetic market:

$$p_{\text{market}} = 0.7 \cdot p_{\text{model}} + 0.3 \cdot p_{\text{seed}} + \varepsilon_{\text{alpha}} + \varepsilon_{\text{noise}}$$

where $\varepsilon_{\text{alpha}} \sim \mathcal{N}(0, 0.03)$ represents information the market has that we don't, and $\varepsilon_{\text{noise}} \sim \mathcal{N}(0, 0.04)$ is microstructure noise. We simulate a 3-cent bid-ask spread on top of this. Getting the synthetic market right took several iterations. Our first version used seed-only pricing and returned 3,684% ROI, which was obviously nonsense.

6.3 Backtest results

Table 3: Trading backtest by threshold (quarter-Kelly, 3-cent spread, \$10K bankroll, 11 seasons).

τ	Trades	P&L	ROI	Win Rate
0.01	388	+\$2,065	+20.6%	63.7%
0.05	375	+\$1,900	+19.0%	62.8%
0.07	280	+\$1,467	+14.7%	62.1%
0.10	165	+\$1,778	+17.8%	67.9%
0.15	63	+\$898	+9.0%	65.3%

All thresholds are profitable. The per-trade ROI goes up at $\tau = 0.10$ compared to $\tau = 0.07$, meaning the medium-confidence trades (edge between 0.07 and 0.10) are net losers. At $\tau = 0.01$, the strategy generated 388 trades across 11 seasons with a Sharpe ratio of 0.88 and only one losing season (2014, $-\$164$). These numbers are probably optimistic because our synthetic market underestimates the efficiency of real Kalshi markets, but the qualitative result, that a model with 0.159 Brier has enough edge to trade profitably at small sizes, seems right.

7 Discussion

7.1 Feature quality vs model complexity

This is the central lesson of the project. We spent two weeks trying fancy architectures on ordinal-rank features and couldn't break 0.196. Then we spent one afternoon scraping Barttorvik and got to 0.169 with the same logistic model. Adding more features and light regularization pushed it to 0.159. The gap between 0.196 and 0.159 came entirely from better data, not better modeling.

In hindsight, the Kaggle competition history predicted this. Past winners use logistic regression or simple Elo with KenPom AdjEM [Pomeroy, 2004]. The exception is Landgraf [2017], who won by predicting competitors' submissions rather than game outcomes. Nobody wins by throwing a more complex model at the same features.

We think the reason is straightforward. The relationship between team quality difference and win probability is close to sigmoidal. Logistic regression nails this functional form by assumption. Nonlinear models (XGBoost, neural nets) have to learn it from data, and with only ~ 670 training games per LOTO fold, they overfit before they converge on something as clean as a sigmoid.

7.2 Where does the signal live?

Barttorvik's net rating difference alone gets 0.169. Adding 16 more features (seed, Four Factors, momentum, coaching, self-computed efficiency, POM rank) brings it to 0.159. That 0.010 improvement is real but modest. Most of the predictive information about NCAA tournament games lives in a single number: the gap in adjusted points per 100 possessions between the two teams.

The ensemble barely moves the needle (0.161 vs 0.159). The market-calibrated ensemble barely moves it further (0.159 vs 0.159). The weight optimizer puts 80–95% on the logistic model in every fold. We believe we're near the ceiling of what team-level aggregate statistics can tell us. Getting below 0.150 would probably require player-level data (injuries, per-player matchup statistics, minutes distributions) or game-specific context (travel distance, rest days, referee tendencies) that we don't have access to.

7.3 Limitations

Our backtesting protocol evaluates on actual tournament games. The Kaggle competition scores on all possible matchups, which means we're graded on games that never happened (e.g., what if the 1-seed in the West played the 3-seed from the East?). Our model still gives reasonable probabilities for these hypothetical matchups because the features are team-level, but we haven't evaluated the calibration of these non-realized predictions.

The trading backtest uses synthetic market prices. Real Kalshi markets are populated by sophisticated quant firms (SIG, DRW, Jane Street) with proprietary data and faster execution. Our edge against them would be smaller than our edge against a synthetic market. We don't know how much smaller.

Our self-computed efficiency (0.225 Brier) is substantially worse than both Barttorvik (0.169) and even ordinal POM rank (0.196). The 50-iteration SOS adjustment doesn't capture home-court effects, tempo adjustments, or player availability. This is a limitation of the approach, not of the data—the raw box scores contain enough information, but extracting it requires a regression framework we didn't have time to build.

7.4 The MoE scaling disaster

The Mixture-of-Experts going from 0.257 to 0.185 (after StandardScaler) deserves a comment because it’s a cautionary tale. Before scaling, LBFGS couldn’t converge because the gradient was dominated by the single largest-scale feature (POM rank difference, range ± 250) while everything else was noise-level. The EM algorithm ran its 10 iterations, the experts never learned anything useful, and the gating network was random. One line of code (adding StandardScaler) fixed everything. We mention this because it’s the kind of bug that doesn’t throw an error—the model “trains” and produces outputs—but the outputs are garbage. Convergence warnings were our only hint.

8 Conclusion

We built a tournament prediction system that achieves 0.159 Brier score on 11 seasons of leave-one-tournament-out evaluation, beating both the seed baseline (~ 0.200) and the Vegas benchmark (~ 0.175). The model is a logistic regression on 21 features dominated by Barttorvik continuous efficiency ratings. We also built and backtested a Kalshi trading strategy that returns 20.6% ROI with a 0.88 Sharpe on synthetic market prices.

The project taught us that in small-sample prediction problems, the features matter more than the model. We implemented a Transformer, a Mixture-of-Experts, gradient boosting, and an ensemble optimizer. The thing that actually worked was plugging a better number into logistic regression.

Code and data are available at <https://github.com/RoddyH17/ncaa-prediction>.

References

- Oliver, D. *Basketball on Paper: Rules and Tools for Performance Analysis*. Potomac Books, 2004.
- Pomeroy, K. KenPom.com: College Basketball Ratings. <https://kenpom.com>, 2004–present.
- Barttorvik. T-Rank: College Basketball Ratings. <https://barttorvik.com>, 2017–present.
- Landgraf, A. March ML Mania 1st Place Solution: Meta-Prediction Strategy. Kaggle Winner Interview, 2017.
- Snowberg, E. and Wolfers, J. Explaining the Favorite–Long Shot Bias: Is it Risk-Love or Misperceptions? *Journal of Political Economy*, 118(4):723–746, 2010.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive Mixtures of Local Experts. *Neural Computation*, 3(1):79–87, 1991.
- Sokol, J., Jank, W., and Harp, S. LRMC: Logistic Regression / Markov Chain for NCAA Basketball. Georgia Institute of Technology, 2006–2026.